

Routing (routing/engine.py) — needs only a positive travel_time_s per edge, nothing else:

RouteGraph.shortest_path(start, goal, edge_cost=lambda edge, t: <your forecast>)

should_switch_route(current_eta_s, candidate_eta_s) # just two numbers

Your SpeedForecast.speed_mps converts to travel_time_s = edge_length_m / speed_mps and plugs straight in. This half of the brain works today with forecasting alone — no perception, no fusion needed.

Signal preemption (corridor/state_machine.py) — needs a PriorityRequest with eta_s, queue_pcu, confidence. eta_s to an intersection is just the ambulance's route ETA up to that node, which you can also derive from forecast + current position — no perception required. So this can work too, once routing produces node-level ETAs, not just edge-level.

What forecasting genuinely cannot give the brain: blockage detection.

RoadObservation.blockage and the fusion override in fusion/engine.py ("Perception override: high-confidence blockage") exist because a historical-average or spatial-temporal baseline will happily predict a plausible speed on a road that is currently physically closed — it has no way to know that. Without perception, an accident/closure scenario (your HIDDEN_ACCIDENT scenario in schema.ScenarioName) will silently degrade to a wrong-but-confident forecast instead of triggering a reroute.

So: forecasting-only is a legitimate, working brain for the common case (route ETA + signal timing under normal traffic), but it is blind to blockages by construction — that gap is what perception exists to close, not a bug in what you've built.

What's your near-term plan — ship the forecast-only brain first and add perception/fusion after, or do you want to stub a placeholder blockage signal (e.g. a manual "accident" control flag, which I see already exists as ControlState.accident) so the hidden-accident scenario doesn't silently fail in the meantime?